

Análise Arquitetural Linha por Linha

Documentação Prática: pagamento.py e test_pagamento.py

1. A Ordem Crítica de Declaração no Python

No ecossistema Python, o interpretador lê e compila o código de cima para baixo de forma estritamente sequencial. Isso impõe regras rígidas de ordenação que estruturam a arquitetura de qualquer software:

- **Biblioteca Sempre no Topo:** O `import requests` deve ser a primeira instrução útil. Se tentarmos colocar qualquer função antes que use o `requests` sem importá-lo no topo, o Python falhará ao tentar resolver o nome global da biblioteca durante o carregamento inicial.
- **Dependências de Invocação (Cadeia Hierárquica):** A função de baixo nível `enviar_para_gateway` é declarada antes de `processar_compra`. Embora o Python permita que funções chamem assinaturas declaradas mais abaixo desde que já estejam todas carregadas na hora da execução, posicionar as funções na ordem em que são consumidas (da infraestrutura para as regras de negócio) garante uma leitura limpa e procedural (Top-Down).
- **Estrutura Lógica de Testes:** No arquivo `test_pagamento.py`, os testes seguem uma ordem didática e de severidade: sucesso total (Caminho Feliz), erro controlado de negócio (Sem Limite) e desastre total de infraestrutura (Timeout).

2. Análise Detalhada de pagamento.py

Código (Linha por Linha)	Explicação Técnica e o Porquê
<pre>import requests</pre>	O que faz: Carrega a biblioteca externa HTTP Requests. Por quê: Permite que nosso software se comunique com servidores web usando os protocolos padrões da internet. Sem isso, não há conexão com nenhuma API externa.
<pre>def enviar_para_gateway(dados_cartao, valor):</pre>	O que faz: Define uma função responsável unicamente pela infraestrutura de comunicação física com o gateway. Por quê: Segue o princípio de responsabilidade única (SOLID). Isolar a chamada de rede em sua própria função permite que o restante do software dependa apenas de um contrato limpo e facilita o isolamento de testes (Mocking).
<pre>url = "https:// api.pagarapido.com.br/v1/transacoes"</pre>	O que faz: Armazena em uma constante local o link (endpoint) oficial do gateway de pagamentos. Por quê: Centraliza o destino para sabermos exatamente para onde os dados sigilosos do cliente serão transmitidos.
<pre>payload = { "cartao": dados_cartao, "valor": valor }</pre>	O que faz: Constrói um objeto do tipo Dicionário estruturando as informações necessárias. Por quê: Traduz os parâmetros recebidos para o formato exato que a API externa exige, preparando o pacote para a conversão em JSON.

Código (Linha por Linha)	Explicação Técnica e o Porquê
<code>response = requests.post(url, json=payload, timeout=10)</code>	O que faz: Envia os dados criptografados via POST e define um limite de espera estrito de 10 segundos. Por quê: Executa a transação financeira real. O parâmetro <code>timeout</code> é obrigatório em sistemas financeiros para impedir que a requisição trave o aplicativo caso o gateway caia.
<code>if response.status_code == 500: raise ConnectionError("Gateway de pagamento indisponível.")</code>	O que faz: Captura falhas de código HTTP 500 vindas do servidor do PagaRápido e joga uma exceção nativa do Python. Por quê: Impede que o sistema continue tentando ler dados de uma resposta que quebrou. Corta o fluxo e sinaliza um erro de conexão imediatamente.
<code>return response.json()</code>	O que faz: Transforma o corpo do texto de resposta recebido da internet em um dicionário Python. Por quê: Facilita a leitura e navegação dinâmica pelas chaves retornadas (como id da transação e status).
<code>def processar_compra(usuario_id, dados_cartao, valor):</code>	O que faz: Declara a função de alto nível que dita as regras de negócio do fechamento do pedido. Por quê: Esta função não quer saber de URLs ou payloads, ela apenas gerencia o comportamento e as mensagens finais entregues ao usuário de acordo com o resultado.
<code>try:</code>	O que faz: Abre uma zona de monitoramento intensivo de erros técnicos. Por quê: Operações de rede falham constantemente. Envolver a chamada em um bloco <code>try</code> impede que uma falha na internet derrube o sistema inteiro, permitindo um tratamento elegante.
<code> resposta_gateway = enviar_para_gateway(dados_cartao, valor)</code>	O que faz: Executa a função de infraestrutura interna que criamos. Por quê: É o ponto de junção das duas camadas. No ambiente real, ela bate na web; no ambiente de testes, o <code>pytest-mock</code> intercepta este exato momento.
<code> status = resposta_gateway.get("status")</code>	O que faz: Recupera o valor da chave "status" usando o método seguro <code>.get()</code>. Por quê: Diferente de usar colchetes (<code>resposta_gateway["status"]</code>), o método <code>.get()</code> evita que o código quebre com <code>KeyError</code> se o gateway esquecer de enviar o campo.
<code> if status == "aprovado": codigo = resposta_gateway.get("transacao_id") return f"Sucesso! Transação {codigo} confirmada."</code>	O que faz: Compara a resposta com a string de validação positiva, extrai a identificação gerada pelo gateway e retorna o texto final de aprovação. Por quê: Conclui o fluxo ideal de sucesso ("Caminho Feliz"), permitindo que o sistema emita o ingresso comprado.

Código (Linha por Linha)	Explicação Técnica e o Porquê
<pre> elif status == "recusado": motivo = resposta_gateway.get("motivo", "Desconhecido") return f"Pagamento recusado. Motivo: {motivo}." </pre>	O que faz: Captura o status de negação financeira, busca a causa fornecida (ou usa um padrão seguro) e gera a string de alerta. Por quê: Trata uma regra comercial sem gerar erros no código. Informa o cliente de forma transparente para que ele troque o cartão ou verifique o limite.
<pre> else: return "Status de pagamento desconhecido." </pre>	O que faz: Atua como uma rede de segurança para respostas não previstas (como 'pendente' ou 'analisando'). Por quê: Evita comportamentos imprevisíveis na interface da aplicação e garante consistência de retorno.
<pre> except ConnectionError: return "Erro no servidor de pagamentos. Tente novamente mais tarde." </pre>	O que faz: Intercepta especificamente o erro gerado quando o servidor parceiro devolve status 500. Por quê: Transforma uma falha técnica complexa em um aviso polido e amigável para o cliente final.
<pre> except requests.exceptions.Timeout: return "Tempo de resposta esgotado. Verifique sua fatura antes de tentar de novo." </pre>	O que faz: Captura a exceção de lentidão extrema da biblioteca requests. Por quê: Protege o cliente contra cobranças em duplicidade, instruindo-o a auditar sua própria fatura antes de tentar submeter a compra novamente.

3. Análise Detalhada de test_pagamento.py

Código de Teste (Linha por Linha)	Explicação Técnica e o Porquê
<pre>import pytest import requests from pagamento import processar_compra</pre>	O que faz: Importa as ferramentas de testes unitários, a biblioteca de rede para referenciar exceções e a função de produção que queremos validar. Por quê: Fornece o ambiente e a matéria-prima necessários para criar as asserções de teste sem poluir ou alterar os arquivos originais.
<pre>def test_processar_compra_aprovada(mocker):</pre>	O que faz: Declara o primeiro teste unitário. A palavra chave <code>mocker</code> injeta a inteligência do <code>pytest-mock</code>. Por quê: Mocks são cruciais no setor financeiro. Eles nos dão superpoderes para simular dinheiro e servidores sem gastar fundos reais nem precisar de internet ativa.
<pre> mock_gateway = mocker.patch("pagamento.enviar_para_gateway")</pre>	O que faz: Intercepta e desativa temporariamente a função <code>enviar_para_gateway</code> localizada no arquivo de produção. Por quê: Garante o isolamento completo. Quando o código rodar, ele não vai acessar a internet; ele vai desviar para o objeto 'dublê' criado pelo patch.
<pre> mock_gateway.return_value = { "status": "aprovado", "transacao_id": "INGRESSO-998877" }</pre>	O que faz: Sobrescreve a saída do nosso dublê, ditando o dicionário exato que ele deve retornar. Por quê: Simula com precisão cirúrgica a aprovação do banco externo para validar se o nosso código principal sabe interpretar caminhos positivos de sucesso.
<pre> resultado = processar_compra(usuario_id=1, dados_cartao={"numero": "1234"}, valor=150.0)</pre>	O que faz: Executa a função de negócios passando argumentos de mentira. Por quê: Força o encadeamento lógico interno a rodar. A função vai achar que está conversando com a API real, mas estará consumindo os dados estáticos do mock.
<pre> assert resultado == "Sucesso! Transação INGRESSO-998877 confirmada."</pre>	O que faz: Executa a asserção matemática/lógica comparando o resultado obtido com o gabarito esperado. Por quê: Se houver qualquer erro de digitação no código de produção ou se o ID não for injetado corretamente na string, o teste quebra na hora, avisando a dupla de desenvolvimento.

Código de Teste (Linha por Linha)	Explicação Técnica e o Porquê
<code>def test_processar_compra_recusada_sem_limite(mocker):</code>	O que faz: Inicia o segundo cenário de teste, focando no fluxo alternativo de negação de crédito. Por quê: Precisamos garantir que o sistema reage de forma estável quando o banco do cliente recusa o pagamento, impedindo telas travadas.
<pre> mock_gateway = mocker.patch("pagamento.enviar_para_gateway") mock_gateway.return_value = { "status": "recusado", "motivo": "Saldo insuficiente" } </pre>	O que faz: Substitui a mesma função, mas configura um dicionário JSON simulando uma rejeição por falta de saldo. Por quê: Isso força o fluxo interno a ignorar o primeiro bloco <code>if</code> e entrar diretamente no bloco <code>elif status == "recusado":</code> de maneira controlada.
<pre> resultado = processar_compra(usuario_id=1, dados_cartao={"numero": "1234"}, valor=5000.0) assert resultado == "Pagamento recusado. Motivo: Saldo insuficiente." </pre>	O que faz: Roda a função e valida se a mensagem capturou dinamicamente o motivo fornecido pelo mock. Por quê: Valida se o sistema trata o cliente com educação e clareza em cenários de problemas com o limite do cartão.
<code>def test_processar_compra_timeout_na_black_friday(mocker):</code>	O que faz: Inicia o teste crítico do Desafio 3, simulando uma queda de infraestrutura sob alta carga. Por quê: É impossível simular de forma barata e segura uma queda real nos servidores do gateway. O mock simula esse caos instantaneamente.
<pre> mock_post = mocker.patch("pagamento.requests.post") </pre>	O que faz: Aplica o patch diretamente na raiz do método <code>post</code> de terceiros da biblioteca <code>requests</code>. Por quê: Em vez de mockar nossa própria função, desligamos a ferramenta de rede interna que ela usa. Isso simula o erro em um nível mais profundo da arquitetura.
<pre> mock_post.side_effect = requests.exceptions.Timeout() </pre>	O que faz: Configura um efeito colateral (<code>side_effect</code>) em vez de um valor de retorno estático. Por quê: Indica ao Python que, ao invés de devolver dados normais, ele deve explodir e disparar uma exceção de interrupção de tempo imediatamente. Isso simula perfeitamente a lentidão severa de rede.

Código de Teste (Linha por Linha)	Explicação Técnica e o Porquê
<pre>resultado = processar_compra(usuario_id=1, dados_cartao={"numero": "1234"}, valor=100.0) assert resultado == "Tempo de resposta esgotado. Verifique sua fatura antes de tentar de novo."</pre>	O que faz: Executa a rotina principal e valida se o bloco de captura except requests.exceptions.Timeout: agiu corretamente. Por quê: Valida o mecanismo de segurança crítico que evita cobranças em duplicidade na carteira do comprador durante eventos de alta concorrência como a Black Friday.